CSE-307 Operating Systems

Learning-Augmented Page Replacement: Classical Algorithms Meet Adaptive Prediction

Student: Md. Mahdi Israk Mahir Student ID: 202414016 Track 1

1. Problem framing

Page replacement decides which resident page should leave memory when a new page is referenced. FIFO and LRU are inexpensive online policies, while Optimal (Belady's) is an offline upper-bound that knows future references. This paper studies whether a small learned policy can use current-state signals to make useful eviction decisions when the access pattern changes after deployment.

2. Implementation and learned component

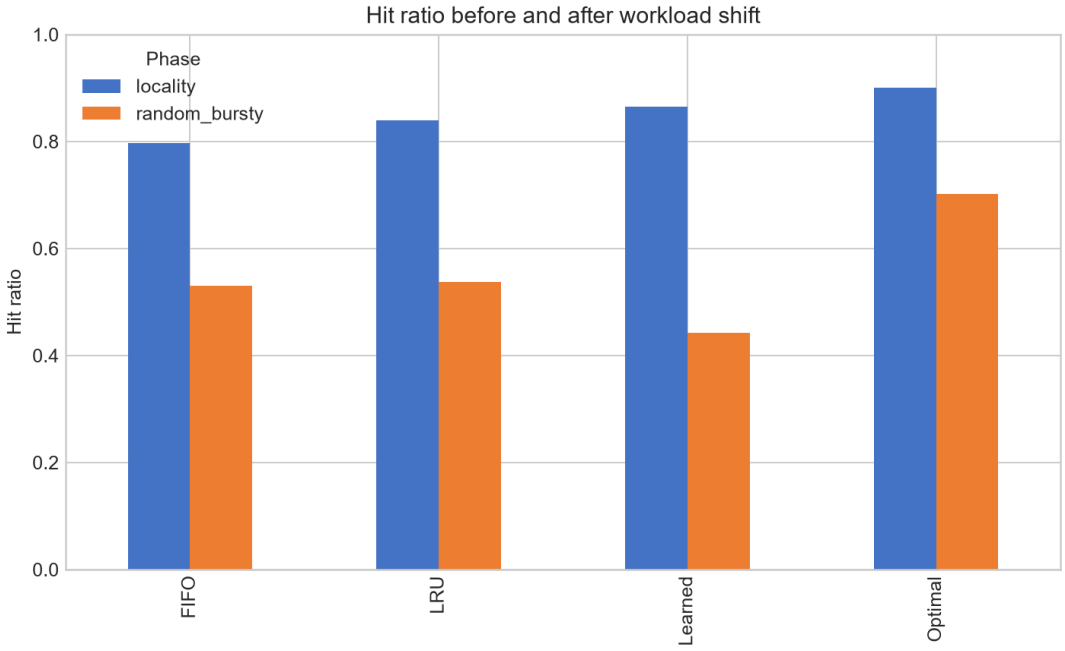
The implementation uses four frames and ten possible pages. FIFO maintains arrival order; LRU tracks the most recent reference position; and Optimal evicts the resident page with the farthest next use. The learned policy is a shallow decision tree trained on a separate synthetic trace. For every resident candidate, it receives only information available at the current miss: page age, access frequency, recency rank, and page identifier. Training labels identify the candidate that Optimal would evict on the training trace. Future references are never supplied as runtime features.

3. Experimental setup

The evaluation trace contains 800 references and is generated with a fixed seed (307). The first 400 references are locality-heavy: pages 0–4 are favored and occasional sequential accesses are inserted. The second 400 references switch to a random/bursty pattern over pages 0–9. Each policy is evaluated separately in both phases using page-fault count and hit ratio.

4. Results

Policy	Locality faults	Locality hit	Shift faults	Shift hit
FIFO	81	79.8%	188	53.0%
LRU	64	84.0%	185	53.8%
Optimal	40	90.0%	119	70.2%
Learned	54	86.5%	223	44.2%


Hit ratio before and after workload shift
Phase
locality
random_bursty
Hit ratio
1.0
0.8
0.6
0.4
0.2
0.0
FIFO
LRU
Learned
Optimal

5. Analysis of the workload shift

The workload change hurts all policies because the second phase has a larger active set and less predictable reuse. LRU falls from 84.0% to 53.8% hit ratio, while FIFO falls from 79.8% to 53.0%. Optimal remains strongest (90.0% to 70.3%) because it has future knowledge and therefore shows the best achievable reference point for this trace.

The learned policy reaches 86.5% in the locality phase, close to Optimal and better than both FIFO and LRU. However, it falls to 44.3% after the shift, the largest absolute degradation in this experiment. The tree learned relationships from a separate trace whose features were dominated by stable reuse. When the trace becomes random and bursty, age and frequency become less reliable predictors of the next use. This result is useful: learning does not automatically guarantee robustness under drift.

A limitation is that the workload is synthetic and the classifier is intentionally small. A stronger adaptive system could detect phase changes online, retrain periodically, use a larger history window, or combine the learned score with an LRU fallback. Those extensions are outside the base Track 1 scope.

6. Conclusion

The classical implementations provide understandable baselines and Optimal supplies an upper bound. The decision tree can improve eviction choices when the workload resembles training conditions, but the shift test shows why operating-system heuristics must be evaluated under distribution change. The main lesson is that a learned component should be monitored and adapted rather than trusted as a permanently correct replacement for a classical policy.

Reproducibility

From the repository root, install the packages in requirements.txt and run **python track1_page_replacement.py**. The command regenerates the trace, CSV table, and charts. The report is generated with **python generate_report.py**.